Taints & Tolerations

First, Remember One Thing

Kubernetes has:

Cluster

  |

  ├── Node 1

  ├── Node 2

  └── Node 3

Pods are created by Kubernetes and are placed on these Nodes.

Normally,

Scheduler

    |

    ▼

Pod

    |

    ▼

Can run on ANY Node

But sometimes we **don't want every Pod to run on every Node.**

This is where **Taints** and **Tolerations** are used.

Real-Life Example 🍕

Imagine there are **3 tables** in a restaurant.

Restaurant

Table 1

Table 2

Table 3

Customers can sit on **any table**.

Now suppose **Table 3** has this sign:

🚫 Reserved for VIP Customers Only

This sign is a **Taint**.

Now imagine two customers.

Customer A

Normal customer.

Cannot sit there.

❌

Customer B

Has a VIP Pass.

Can sit there.

✅

The **VIP Pass** is a **Toleration**.

Kubernetes Example

Restaurant = Kubernetes Cluster

Tables = Nodes

Customers = Pods

VIP Sign = Taint

VIP Pass = Toleration

What is a Taint?

Simple Definition

A Taint is a rule or label placed on a Node that tells Kubernetes not to schedule normal Pods on that Node.

Easy Memory

🚫 **Taint = "No Entry" sign on a Node**

Example

Node 1

Normal Node

Pods can run.

Node 2

🚫 GPU ONLY

Normal Pods cannot run.

Why Use Taints?

Suppose you have a very powerful server.

Node 2

64 GB RAM

GPU Installed

You want only AI applications to use it.

You don't want small applications wasting its resources.

So you apply a **Taint**.

Example

Before Taint

Node 1

Pod A

Pod B

Pod C

Node 2

Pod D

Pod E

Everything runs everywhere.

After Taint

Node 2

🚫 No Normal Pods

Only special Pods can run there.

What is a Toleration?

Simple Definition

A Toleration is a permission given to a Pod so it can run on a Node that has a matching Taint.

Easy Memory

🔑 **Toleration = Special Permission**

Example

Node has:

🚫 GPU ONLY

Normal Pod

No Permission

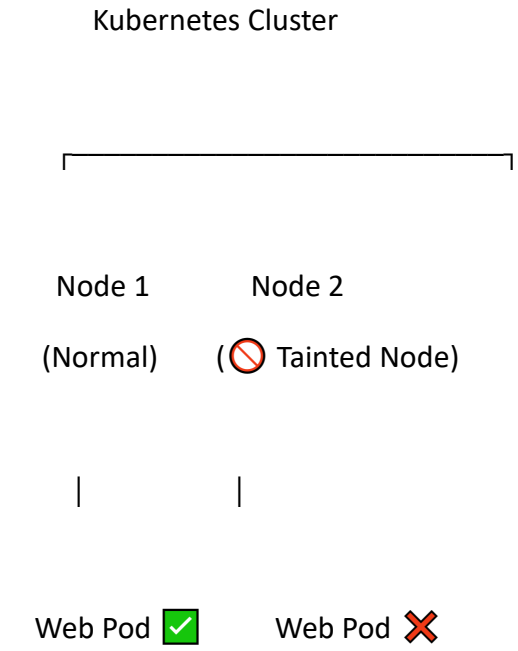
Cannot run.

AI Pod

Has Permission

Runs successfully.

Visual Diagram



AI Pod ✅

(Has Toleration)

Another Easy Example 🏫

Think of a school.

There are two classrooms.

Classroom A

Everyone can enter.

Classroom B

🚫 Teachers Only

This is a **Taint**.

Students

Student A

Cannot enter.

Teacher

Has Teacher ID Card

Can enter.

The ID Card is the **Toleration**.

Kubernetes Scheduling Process

Suppose Scheduler wants to place a Pod.

Step 1

Scheduler checks Node.

Is Node Tainted?

↓

YES

↓

Check Pod.

Does Pod have Toleration?

↓

YES

↓

Schedule Pod.

If NO

↓

Don't Schedule.

Example

Suppose

Cluster

Node 1

Node 2

Node 2 has

🚫 Database Only

Now

Web Pod

No Toleration

Cannot run.

Database Pod

Has Database Toleration

Runs successfully.

Kubernetes Commands

Add Taint

kubectl taint nodes node1 special=gpu:NoSchedule

Meaning

Node

↓

special = gpu

↓

NoSchedule

This tells Kubernetes:

Don't schedule normal Pods here.

View Taints

kubectl describe node node1

Output

Taints:

special=gpu:NoSchedule

Remove Taint

kubectl taint nodes node1 special=gpu:NoSchedule-

Notice

-

removes the taint.

Add Toleration to Pod

Inside Pod YAML

```
tolerations:
- key: "special"
  operator: "Equal"
  value: "gpu"
  effect: "NoSchedule"
```

This Pod is now allowed to run on the GPU Node.

Taint Effects

There are **3 effects**.

---

1. NoSchedule ⭐

Meaning

Don't place new Pods here.

Without Toleration

❌ Cannot run.

With Toleration

✅ Can run.

---

2. PreferNoSchedule

Meaning

Avoid using this Node.

Kubernetes tries another Node first.

If necessary,

it may still use this Node.

Think

⚠️ Avoid if possible.

---

3. NoExecute

Meaning

Don't allow Pods without permission to stay.

New Pods

❌ Cannot enter.

Existing Pods

❌ Removed.

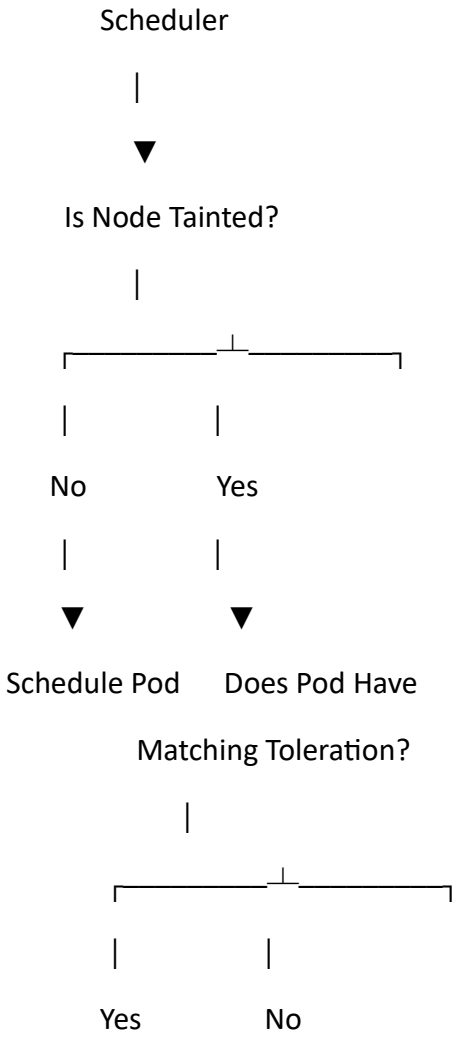
Pods with Toleration

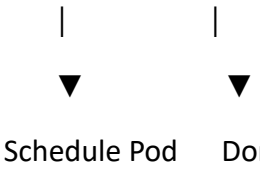
✅ Stay.

Difference Between Taint and Toleration

Taint	Toleration
Applied on **Nodes**	Applied on **Pods**
Restricts Pod scheduling	Allows Pod scheduling
Acts like a "No Entry" sign	Acts like a permission pass
Protects special Nodes	Lets special Pods use those Nodes

Complete Flow





Easy Memory Trick ⭐

Node

|

├— Taint 🚫

|

| "Only Special Pods"

|

▼

Pod

|

├— Toleration 🔑

|

| "I Have Permission"

|

▼

Scheduler

|

├— Permission? ✅ → Run Pod

|

└— No Permission? ❌ → Reject Pod

KCNA Revision Notes

Taints

- Applied on **Nodes**.
- Restrict which Pods can run.
- Used to reserve special Nodes.
- Example: GPU, Database, High-memory Nodes.

Tolerations

- Applied on **Pods**.
- Allow Pods to run on tainted Nodes.
- Must match the Node's taint.

Taint Effects

- **NoSchedule** → Block new Pods.
- **PreferNoSchedule** → Try to avoid the Node.
- **NoExecute** → Remove Pods without a matching toleration.

One-Line Memory Trick

- 🚫 **Taint = Rule on a Node ("Keep Out").**
- 🔑 **Toleration = Permission on a Pod ("I Can Enter").**

The most important sentence to remember for KCNA:

Taints are applied to Nodes to restrict Pod scheduling, while Tolerations are applied to Pods to allow them to run on Nodes with matching Taints.

Here are the **most important Kubernetes commands** related to **Taints and Tolerations** for **KCNA beginners**.

---

1. View All Nodes

kubectl get nodes

Purpose: Displays all Nodes in the cluster.

Example Output:

NAME STATUS ROLES AGE

node1 Ready control-plane 10d

node2 Ready worker 10d

---

2. View Taints on a Node ⭐

kubectl describe node node1

Or

kubectl describe nodes node1

Purpose: Shows detailed information about the Node.

Look for:

Taints:

special=gpu:NoSchedule

If you see:

Taints: <none>

It means the Node has no taints.

---

3. Add a Taint to a Node ⭐

Syntax

kubectl taint nodes <node-name> key=value:effect

Example

kubectl taint nodes node1 special=gpu:NoSchedule

Meaning:

- Node = node1
- Key = special

- Value = gpu
- Effect = NoSchedule

Result:

Only Pods with the matching toleration can run on this Node.

---

4. Remove a Taint ⭐

Syntax

kubectl taint nodes <node-name> key=value:effect-

Notice the **-** at the end.

Example

kubectl taint nodes node1 special=gpu:NoSchedule-

Result:

The taint is removed from the Node.

---

5. View All Pods

kubectl get pods

---

6. View Pods on All Namespaces

kubectl get pods -A

or

kubectl get pods --all-namespaces

---

7. View Pod Details

kubectl describe pod <pod-name>

Example

kubectl describe pod nginx

This shows:

- Pod Status
- Node Name
- Events
- Tolerations
- Container Details

Look for:

Tolerations:

---

8. Check Which Node a Pod is Running On

kubectl get pods -o wide

Example Output

NAME READY STATUS NODE

nginx 1/1 Running node2

The **NODE** column tells you where the Pod is running.

---

9. View Tolerations of a Pod

kubectl describe pod <pod-name>

Example

kubectl describe pod nginx

Look for:

Tolerations:

Example Output

Tolerations:

special=gpu:NoSchedule

---

10. Create a Pod with a Toleration

Example YAML

```
apiVersion: v1
kind: Pod

metadata:
  name: nginx

spec:
  tolerations:
  - key: "special"
    operator: "Equal"
    value: "gpu"
    effect: "NoSchedule"

  containers:
  - name: nginx
    image: nginx
```

Apply it:

```
kubectl apply -f pod.yaml
```

---

11. Delete the Pod

```
kubectl delete pod nginx
```

---

12. Edit a Resource

```
kubectl edit pod nginx
```

or

kubectl edit deployment nginx

---

13. Export Pod YAML

kubectl get pod nginx -o yaml

Useful for viewing tolerations and other Pod settings.

---

14. Export Node YAML

kubectl get node node1 -o yaml

Useful for viewing taints.

Most Important KCNA Commands ⭐

Command	Purpose
kubectl get nodes	List all Nodes
kubectl describe node node1	View Node details and taints
kubectl taint nodes node1 special=gpu:NoSchedule	Add a taint
kubectl taint nodes node1 special=gpu:NoSchedule-	Remove a taint
kubectl get pods	List Pods
kubectl get pods -o wide	Show the Node where each Pod is running
kubectl describe pod nginx	View Pod details and tolerations
kubectl apply -f pod.yaml	Create a Pod from a YAML file
kubectl delete pod nginx	Delete a Pod
kubectl get pod nginx -o yaml	Display the Pod's YAML configuration

---

Easy Memory Trick ⭐

Check Nodes

kubectl get nodes

Check Taints

kubectl describe node node1

Add Taint

kubectl taint nodes node1 special=gpu:NoSchedule

Remove Taint

kubectl taint nodes node1 special=gpu:NoSchedule-

Check Pods

kubectl get pods

Check Pod Details

kubectl describe pod nginx

Check Pod's Node

kubectl get pods -o wide

Create Pod

kubectl apply -f pod.yaml

Delete Pod

kubectl delete pod nginx

These are the commands you are most likely to use and be asked about in **KCNA labs and interviews**.